

MODUL 322

SupportBot AI

KI-gestützter Kundenservice-Chat

Technologie: Blazor Web App · .NET 8 · Interactive Server · Gemini 2.5 Flash

Name: [VORNAME NACHNAME EINTRAGEN]

Klasse: [KLASSE EINTRAGEN]

Abgabedatum: [DATUM EINTRAGEN]

Repository: [GITHUB-LINK EINTRAGEN]

TechShop AG ist fiktiv. Produkt-, Bestell- und Kundendaten dienen ausschliesslich dieser Modulprüfung.

Inhalt und Abgabestatus

Gesamtdokumentation gemäss HZ1–HZ5

Kapitel	Inhalt	Status
1	Nutzungsanalyse und Personas (HZ1)	ausgearbeitet
2	Entwurf und KI-Interaktionskonzept (HZ2)	ausgearbeitet
3	Implementierung und kritische Komponente (HZ3)	Code + Bericht
4	Usability-Prüfung und Trust (HZ4)	Peer-Daten offen
5	Barrierefreiheit und Audit (HZ5)	Audit offen
6	Qualitätssicherung	Testprotokoll offen
7	Reflexion und Quellen	ausgearbeitet

Vor Abgabe zwingend: Zwei echte Peer-Tests durchführen, SUS/Trust-Werte eintragen, drei daraus abgeleitete Änderungen im Code umsetzen, App-Screenshots sowie Lighthouse-/axe-Nachweis ergänzen.

Projektartefakte

- Visual-Studio-Solution SupportBotAI.sln und vollständiger Blazor-Quellcode
- Balsamiq-Projekt mit 15 Boards, zwei Chatvarianten und Zustandsflows
- JSON-Persistenz, echter Gemini-Stream, Fehler-/Timeout-Demo, Feedback und Eskalation
- Testvorlagen, WCAG-Checkliste, README, Präsentation und diese Dokumentation

1. Nutzungsanalyse

HZ1 · Umfeld, Risiken und Hauptaufgaben

Die fiktive TechShop AG erhält viele repetitive Anfragen zu Bestellstatus, Retouren und Produktkompatibilität. Der SupportBot AI soll schnell helfen, ohne einen Menschen vorzutäuschen. Bei fehlendem Kontext, Fehlern oder Unzufriedenheit bleibt die Übergabe an einen Mitarbeitenden jederzeit möglich.

Dimension	Beobachtung	UI-Konsequenz
Gerät	Desktop, Tablet, Smartphone	responsive Browserlösung
Situation	unterbrechungsanfällig, teils unter Zeitdruck	persistenter Verlauf, klare Zustände
Erfahrung	digital versiert bis wenig geübt	Schnellfragen, klare Labels
KI-Vertrauen	neugierig bis skeptisch	Textlabel, Grenzen, Feedback
Einschränkung	Screenreader, Seh- oder Motorikeinschränkung	Tastatur, aria-live, Kontrast, Sprache
Datenschutz	Text geht an externen LLM-Dienst	Datenminimierung und Warnung

Hauptaufgaben

1. Frage frei eingeben oder Schnellfrage auswählen.
2. Warte-, Streaming-, Erfolgs- oder Fehlerzustand verstehen.
3. Antwort prüfen, bewerten, erneut versuchen oder abbrechen.
4. Bei fehlender Lösung mit Kontext an menschlichen Support übergeben.
5. Frühere Verläufe und persönliche Accessibility-Einstellungen nutzen.

1.1 Drei komplementäre Personas

Differenzierte Bedürfnisse statt eines Durchschnittsnutzers

Lea Meier – KI-skeptische Power-Userin

34 · Projektleiterin · Notebook und Smartphone

„Ich nutze KI gern – solange klar ist, was sie weiss und was nicht.“

Abgeleitete Bedürfnisse: KI-Kennzeichnung, Kontextangaben, Unsicherheit, Feedback, Retry/Abbruch und dauerhafte Eskalation

Peter Keller – wenig geübter Nutzer

61 · Verkauf · älterer Laptop

„Sag mir einfach, was ich als Nächstes tun muss.“

Abgeleitete Bedürfnisse: Schnellfragen, ausgeschriebene Aktionen, Formatbeispiele, Bestätigungen und verständliche Fehler

Nora Frei – blinde Screenreader-Nutzerin

29 · Juristin · Chrome, Tastatur und Screenreader

„Wenn etwas neu erscheint, muss mein Screenreader wissen, dass es da ist.“

Abgeleitete Bedürfnisse: Semantik, aria-live, Fokusmanagement, sichtbarer Fokus, KI-Textlabel, Spracheingabe und Sprachausgabe

1.2 Priorisierte Anforderungen I

Must-Anforderungen mit überprüfbaren Akzeptanzkriterien

ID	Anforderung	Persona	Akzeptanzkriterium
R01	Frage frei senden	alle	leer blockiert; 2000 Zeichen; Strg+Enter
R02	Wartezustand verstehen	alle	Ladeindikator und aria-busy bis Streamende
R03	KI eindeutig erkennen	Lea	jede KI-Antwort trägt KI-GENERIERT
R04	Grenzen offenlegen	Lea	keine erfundenen Fakten; Eskalation bei Unsicherheit
R05	nach Fehler weiterkommen	Peter	verständlicher Fehler, Retry und Übergabe
R06	menschliche Hilfe	alle	Eskalation im Normal- und Fehlerzustand
R07	vollständige Übergabe	Support	Name, TS-12345, Anliegen, Dringlichkeit validiert
R08	Chat-Snapshot übergeben	alle	sichtbares Opt-in; gespeicherte Momentaufnahme
R09	jede KI-Antwort bewerten	Lea	Ja/Nein plus optionaler Kommentar

Priorisierung: Diese neun Anforderungen sind Must, weil sie direkt die Kernaufgabe, das KI-Vertrauen oder die sichere Eskalation betreffen.

1.3 Priorisierte Anforderungen II

Barrierefreiheit, Kontrolle und Komfort

ID	Anforderung	Prio	Akzeptanzkriterium
R10	neue Inhalte ankündigen	Must	role=log, aria-live, Fokus auf neue Antwort
R11	vollständig per Tastatur	Must	Skip-Link, DOM-Reihenfolge, focus-visible
R12	Spracheingabe	Must	Chrome Web Speech API, de-CH, Transkript im Feld
R13	Darstellung anpassen	Must	Textgrösse und Kontrast lokal gespeichert
R14	Schnellfragen	Should	drei klar benannte Beispiele
R15	Verläufe finden	Should	Liste, Suche, Statusfilter und Fortsetzen
R16	Antwort abbrechen	Should	kontrollierter Abbruch ohne Verlust der Frage
R17	Fehler demonstrieren	Should	einmaliger reproduzierbarer Demo-Timeout
R18	Antwort vorlesen	Could	optionale Sprachausgabe mit Tempo

Vertrauensprinzip

Vertrauen entsteht hier nicht durch eine menschenähnliche Inszenierung, sondern durch Vorhersagbarkeit, Ehrlichkeit und Kontrolle: Modell und KI-Rolle sind sichtbar, Unsicherheit wird benannt und Nutzende können abbrechen, bewerten, erneut versuchen oder eskalieren.

1.4 Ergonomische Dialoggestaltung

Anwendung ausgewählter Grundsätze der DIN EN ISO 9241-110

Grundsatz	Umsetzung
Aufgabenangemessenheit	Fokus auf Bestellung, Retoure und Produktfrage; keine fachfremden Funktionen
Selbstbeschreibungsfähigkeit	ausgeschriebene Aktionen, Feldhilfe, Status, Bestätigung und Datenschutztext
Steuerbarkeit	Abbruch, Retry, Fortsetzen, Filter, Einstellungen und jederzeitige Eskalation
Erwartungskonformität	bekannte Chatmetapher, konsistente Navigation und Aktionshierarchie
Fehlertoleranz	Validierung, blockierte Leereingabe, Timeout ohne Verlust, Löschbestätigung
Individualisierbarkeit	Textgrösse, Kontrast, Sprach-Ein/-Ausgabe und Ankündigungen
Lernförderlichkeit	Schnellfragen, Beispiele und wiederkehrende Positionen

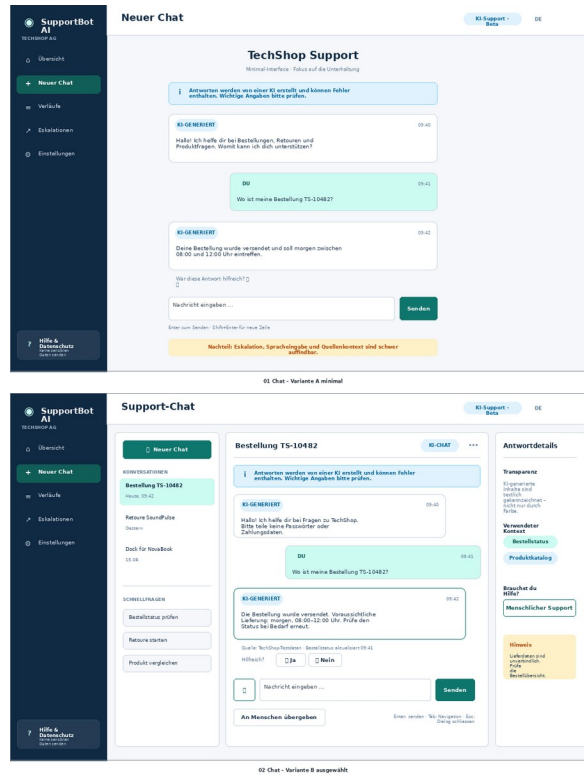
Gestaltungsziel: So wenig wie möglich, aber so viel Transparenz und Kontrolle wie für eine KI-Oberfläche nötig.

Allgemeine Richtlinien

- Konsistente Fluchtlinien, Abstände, Schriftstufen und Farben
- Status nie ausschliesslich über Farbe oder Symbol vermitteln
- Sinnvolle Standardwerte: aktueller Chat, normale Dringlichkeit, Verlauf anhängen
- Primäre Handlung deutlich, destruktive Handlungen mit Warnung

2. Entwurf des Chatfensters

HZ2 · Zwei Varianten, begründete Auswahl



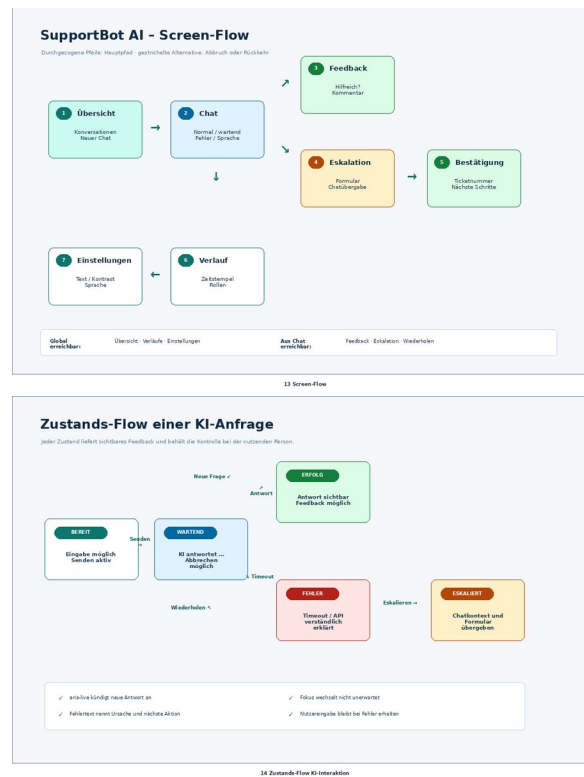
Variante A – minimal | Variante B – reichhaltig, ausgewählt

Kriterium	Variante A	Variante B
Orientierung	sehr reduziert	Verläufe und Schnellfragen
Transparenz	Grundkennzeichnung	KI-Label, Kontext, Antwortdetails
Kontrolle	wenige direkte Aktionen	Feedback, Abbruch, Retry, Eskalation
Risiko	wichtige Grenzen übersehbar	höhere Dichte auf kleinen Geräten

Entscheid: Variante B wurde umgesetzt. Auf schmalen Breiten fallen die Seitenspalten weg; der Kern verhält sich dann ähnlich kompakt wie Variante A.

2.1 Screen- und Zustandsflow

Alle normalen, wartenden, fehlerhaften und eskalierten Wege



Screen-Flow der sechs Pflichtseiten | Zustands-Flow einer KI-Nachricht

KI-spezifische Muster

- KI wird im Produktnamen, im Chat-Badge und an jeder Antwort textlich bezeichnet.
- Warten wird als Nachricht mit Ladeindikator und aria-busy dargestellt.
- Fehler bewahren die Nutzerfrage und bieten Retry sowie Eskalation.
- Feedback ist nach jeder erfolgreichen KI-Antwort inline erreichbar.
- Confidence-Prozentwerte wurden verworfen, weil keine kalibrierte Modellwahrscheinlichkeit vorliegt.

3. Implementierung

HZ3 · Blazor Web App, .NET 8, Interactive Server

Technologiewahl: Blazor bietet semantisches HTML, aria-live, Web Speech API, serverseitig geschützte Secrets und eine direkte Streaming-Darstellung im Browser.

Ebene	Verantwortung	Beispiele
Razor UI	Darstellung und Interaktion	Chat, Verlauf, Eskalation, Feedback, Einstellungen
Models	Zustand und Validierung	Conversation, ChatMessage, FeedbackEntry
Services	KI und Persistenz	GeminiChatService, JsonAppDataStore
JS-Interop	Browserfunktionen	Speech Recognition, Speech Synthesis, Fokus
Konfiguration	Modell und Secret	appsettings, Visual Studio User Secrets

Sechs Pflichtscreens

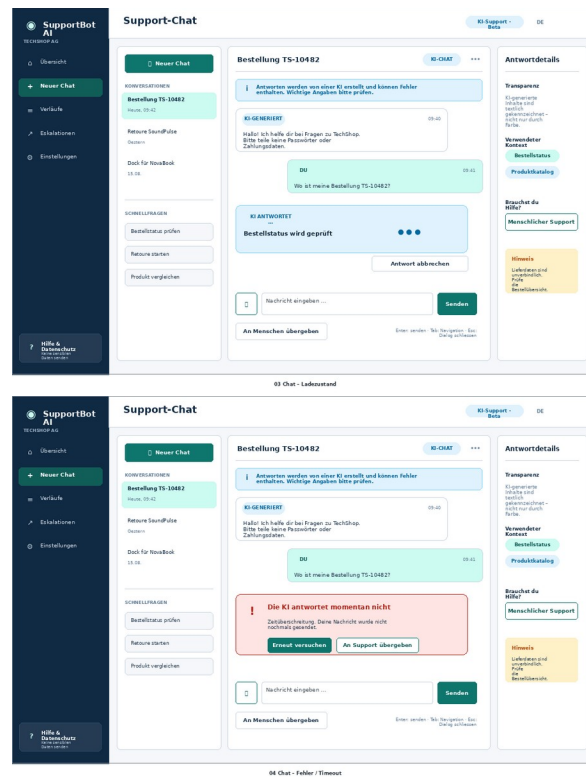
Übersicht/Start, Chat-Fenster, Konversationsdetail, Eskalation, Feedback sowie Einstellungen/Barrierefreiheit sind als eigenständige Razor-Seiten vorhanden. Ergänzend gibt es filterbare Verläufe und eine Eskalationsübersicht.

Datenhaltung

Chats, Feedback und Eskalationen werden lokal in JSON gespeichert. Ein SemaphoreSlim schützt parallele Schreibvorgänge; eine temporäre Datei reduziert das Risiko unvollständiger Writes. Laufzeitdaten und Secrets sind vom Git-Repository ausgeschlossen.

3.1 Kritische Komponente: KI-Streaming

Robuste Zustandsverwaltung trotz variabler Antwortzeit



Wartend/Streaming | Timeout mit Retry und Übergabe

1. Nutzertext vor dem API-Aufruf persistent speichern.
2. Eine KI-Nachricht mit `IsStreaming=true` anlegen.
3. Textteile aus `IAsyncEnumerable<string>` an dieselbe Nachricht anhängen.
4. Mit `WaitAsync(timeoutToken)` einen echten UI-Timeout erzwingen.
5. Im finally-Block Busy-State, Ressourcen, Demo-Schalter und Speicherung abschliessen.

Invariante: Nach Erfolg, Fehler oder Abbruch bleibt die Nutzerfrage erhalten und die Oberfläche wird wieder bedienbar.

3.2 Fehlerbehandlung, Kontext und Datenschutz

Keine technischen Sackgassen und keine vorgetäuschte Gewissheit

Fall	Feedback	Wiederherstellung
API-Key fehlt	klare Konfigurationsmeldung	User Secrets ergänzen, Retry
Timeout	Antwort kam nicht rechtzeitig; Frage bleibt	Retry oder menschlicher Support
Nutzerabbruch	Antwort wurde abgebrochen	erneut starten oder weiterchatten
API/Netzwerk	Dienst momentan nicht erreichbar	Retry oder Eskalation
Formular ungültig	feldnahe Korrekturtexte	Eingabe direkt korrigieren
JSON beschädigt	ungültige Datei lokal gesichert	Seed-Daten neu anlegen

Begrenzter fiktiver Kontext

Der System-Prompt enthält fünf Produkte, drei Testbestellungen und vier Richtlinien. Das Modell darf keine weiteren Bestellungen, Preise oder Liefertermine erfinden und soll bei fehlendem Wissen offen eskalieren.

Datenschutz

Nutzereingaben werden zur Antwort an Gemini übermittelt. Deshalb nennt die Oberfläche ausdrücklich, dass keine Passwörter oder vollständigen Zahlungsdaten eingegeben werden dürfen. Für eine produktive Lösung wären zusätzlich Rechtsgrundlage, Aufbewahrung, Zugriffsschutz, Inhaltsfilter und ein Vertrag mit dem Auftragsbearbeiter zu klären.

4. Usability-Überprüfung: Walkthrough

HZ4 · Variantenvergleich anhand zweier Szenarien

Szenario	Prüfschritte	erwarteter Vorteil B
Bestellstatus TS-10482	Chat starten; Frage formulieren; Warten erkennen; KI-Herkunft und Ergebnis verstehen; bewerten	Schnellfrage, Modellstatus, Kontext und Feedback sichtbar
TS-99999 eskalieren	Grenze erkennen; Übergabe finden; Pflichtfelder verstehen; Verlauf prüfen; Bestätigung verstehen	Eskalation permanent sichtbar; Zusammenfassung vor Absenden

Leitfragen pro Schritt

1. Wird das richtige Ziel ohne zusätzliche Anleitung erkannt?
2. Ist die passende Aktion sichtbar und verständlich beschriftet?
3. Ist die Systemreaktion nach der Aktion nachvollziehbar?
4. Kann ein Irrtum mit minimalem Aufwand korrigiert werden?

Protokoll: Die vollständigen 0–2-Bewertungstabellen liegen unter docs/testing/01_Walkthrough.md und werden nach der Durchführung hier zusammengefasst.

4.1 Peer-Test, SUS und Vertrauen

Zwei echte Drittpersonen • Ergebnisse vor Abgabe ergänzen

AUSSTEHEND: Diese Seite darf erst nach zwei echten Tests finalisiert werden. Keine Werte erfinden.

Kennzahl	TP-01	TP-02	Mittelwert
SUS-Score (0–100)	[eintragen]	[eintragen]	[berechnen]
T1 KI jederzeit erkennbar (1–5)	[]	[]	[]
T2 Grenzen verständlich (1–5)	[]	[]	[]
T3 menschliche Hilfe klar (1–5)	[]	[]	[]
Aufgaben ohne Hilfe	[] / 6	[] / 6	[] %

SUS-Auswertung

Für ungerade Fragen wird Antwort minus 1 gerechnet, für gerade Fragen 5 minus Antwort. Die Summe der zehn Beiträge wird mit 2.5 multipliziert. Die drei Trust-Fragen werden separat ausgewertet und nicht in den SUS-Score gemischt.

Beobachtungen und Zitate

1. [Konkretes Verhalten oder anonymisiertes Zitat eintragen]
2. [Konkretes Verhalten oder anonymisiertes Zitat eintragen]
3. [Konkretes Verhalten oder anonymisiertes Zitat eintragen]

4.2 Abgeleitete Optimierungen

Mindestens drei testbasierte Änderungen nachweisbar umsetzen

Arbeitsregel: Nur Änderungen als Peer-Test-Ergebnis ausgeben, die im Protokoll beobachtet und anschliessend im Code umgesetzt wurden.

ID	Befund	Änderung	Datei / Nachprüfung
O1	[Testbefund]	[konkrete Änderung]	[Datei] · [bestanden/offen]
O2	[Testbefund]	[konkrete Änderung]	[Datei] · [bestanden/offen]
O3	[Testbefund]	[konkrete Änderung]	[Datei] · [bestanden/offen]

Geeignete kleine Anpassungen – nur bei passendem Befund

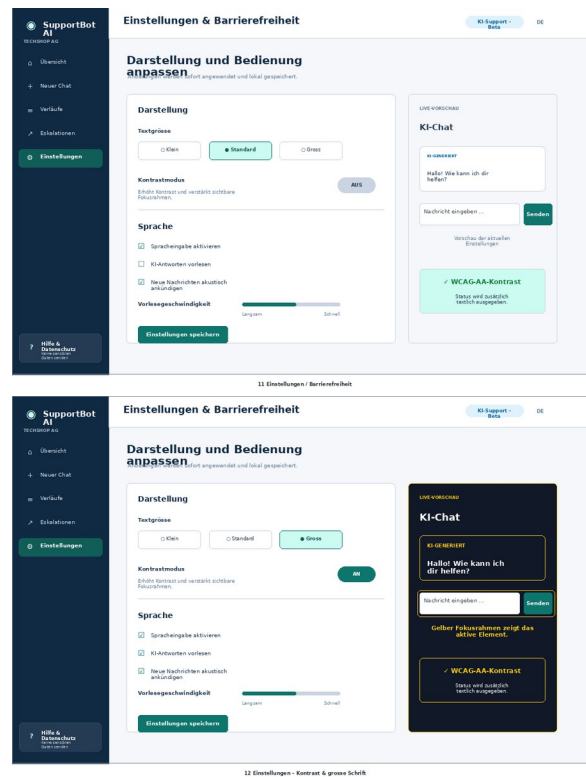
- Aktionslabel verständlicher formulieren oder Formatbeispiel deutlicher platzieren.
- Fokus nach einer KI-Antwort oder Fehlermeldung weiter verbessern.
- Warnhinweis kürzen, Kontrast erhöhen oder Touch-Zielfläche vergrössern.
- Schnellfrage oder Eskalationspfad eindeutiger benennen.

Vorher/Nachher-Nachweis

Für jede Optimierung einen kurzen Vorher/Nachher-Satz, den Commit oder die betroffene Codezeile und das Resultat einer kurzen Nachprüfung dokumentieren.

5. Barrierefreiheit

HZ5 · Wahrnehmbar, bedienbar, verständlich und robust



Accessibility-Einstellungen | Hoher Kontrast und grosse Schrift

Anforderung	Umsetzung
dynamische Inhalte	role=log, aria-live, aria-busy, globaler Announcer
Tastatur und Fokus	native Controls, Skip-Link, focus-visible, Fokus auf neue Antwort
Spracheingabe	Chrome Web Speech API, de-CH, Interim- und Endresultat
Sprachausgabe	optionale Speech Synthesis mit regelbarem Tempo
Kontrast/Text	High-Contrast-Modus, drei Textgrössen, gespeicherte Präferenz
KI-Kennzeichnung	Textlabel KI-GENERIERT, nicht nur Farbe oder Position

5.1 WCAG-2.1-Check und automatisierter Audit

Quellcode-Nachweis plus manuelle Prüfung der laufenden App

Thema	Quellcode-Status	manueller Nachweis
1.3.1 Semantik	implementiert	Lighthouse/axe
1.4.1 nicht nur Farbe	implementiert	visuelle Prüfung
1.4.3 Kontrast	CSS definiert	Kontrastwerte/Audit
1.4.10 Reflow	Breakpoints vorhanden	320 CSS px
2.1.1 Tastatur	native Controls	vollständiger Rundgang
2.4.1 Skip-Link	implementiert	Tastaturtest
2.4.7 Fokus sichtbar	focus-visible	Tastaturtest
3.3 Fehlerhilfe	Validation + Retry	Formular- und Timeouttest
4.1.2 Name/Rolle/Wert	ARIA und native Elemente	axe + Screenreader
4.1.3 Statusmeldungen	aria-live/status/alert	Narrator oder NVDA

AUDIT EINTRAGEN: Lighthouse Accessibility: [__/100] · axe kritische Fehler: [__] · Browser/Datum: [eintragen]. Screenshot der Resultate ergänzen.

6. Reflexion und Quellen

Was KI-gestützte Oberflächen besonders anspruchsvoll macht

Reflexion

Die grösste Herausforderung war nicht die Texteingabe, sondern der kontrollierte Umgang mit Unsicherheit. Eine klassische Oberfläche liefert meist deterministische Resultate; eine LLM-Antwort kann langsam, unvollständig oder falsch sein. Deshalb wurden Ladezustand, klare KI-Kennzeichnung, Retry, Abbruch, Feedback und menschliche Eskalation als Teil der Kernfunktion behandelt.

Besonders wichtig war die Entscheidung gegen eine erfundene Confidence-Prozentzahl. Sie würde Genauigkeit suggerieren, die das verwendete Modell nicht zuverlässig liefert. Stattdessen zeigt die App den verwendeten fiktiven Kontext und nennt Grenzen im Text. Für eine nächste Version würde ich serverseitige Telemetrie ohne Inhaltsdaten, eine echte Wissensdatenbank mit zitierbaren Dokumenten und eine datenschutzrechtlich geprüfte Lös- und Aufbewahrungsstrategie ergänzen.

Der Versand von Nutzertext an eine externe Gemini API bleibt eine Datenschutzfrage. Die Demo verwendet nur fiktive Daten und warnt vor Passwörtern und Zahlungsdaten. Für Produktion wären Einwilligung beziehungsweise Rechtsgrundlage, Datenstandort, Auftragsbearbeitung, Zugriffskontrolle und Missbrauchsschutz vorab zu klären.

Quellen

- Google Gen AI .NET SDK: <https://googleapis.github.io/dotnet-genai/>
- Gemini API – Modelle: <https://ai.google.dev/gemini-api/docs/models>
- Gemini API – Preise: <https://ai.google.dev/gemini-api/docs/pricing>
- ASP.NET Core Blazor: <https://learn.microsoft.com/aspnet/core/blazor/>
- WCAG 2.1: <https://www.w3.org/TR/WCAG21/>
- System Usability Scale: <https://www.usability.gov/how-to-and-tools/methods/system-usability-scale.html>

Finalisierung: Name, Datum, GitHub-Link, echte Testergebnisse, drei Optimierungen, App-Screenshots und Audit-Nachweis ergänzen; danach den Entwurfsvermerk aus Dateiname und Titel entfernen.